

EXPERIMENT-11

Write a program for Credit Score Classification using a suitable machine learning algorithm.

Code:

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

print("=" * 70)
print("      CREDIT SCORE CLASSIFICATION")
print("=" * 70)

uploaded = files.upload()
filename = list(uploaded.keys())[0]
data = pd.read_csv(filename)
print("\nCREDIT SCORE DATASET")
print("-" * 70)
print(data.head(15))
print("\nTotal Records :", len(data))

# Convert categorical columns
data = data.copy()

for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = LabelEncoder().fit_transform(data[col].astype(str))

# Find target column
target_col = None

for col in data.columns:
```

```

if 'credit' in col.lower() and 'score' in col.lower():
    target_col = col
    break
if target_col is None:
    target_col = data.columns[-1]
print("\nTarget Column :", target_col)
X = data.drop(columns=[target_col])
y = data[target_col]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
print("\nBuilding Decision Tree...")
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("\nMODEL TRAINING COMPLETED")
print("\nActual Values:")
print(list(y_test))
print("\nPredicted Values:")
print(list(prediction))
print("\nAccuracy:")
print(round(accuracy_score(y_test, prediction) * 100, 2), "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, prediction))
print("\nCredit Score Classification Completed Successfully.")

```

Sample Input:

```

... =====
                CREDIT SCORE CLASSIFICATION
=====
Choose Files | Credit Scor... Dataset.csv
Credit Score Classification Dataset.csv(text/csv) - 9001 bytes, last modified: 8/10/2026 - 100% done
Saving Credit Score Classification Dataset.csv to Credit Score Classification Dataset.csv

```

Sample Output:

*** CREDIT SCORE DATASET

	Age	Gender	Income	Education	Marital Status	\
0	25	Female	50000	Bachelor's Degree	Single	
1	30	Male	100000	Master's Degree	Married	
2	35	Female	75000	Doctorate	Married	
3	40	Male	125000	High School Diploma	Single	
4	45	Female	100000	Bachelor's Degree	Married	
5	50	Male	150000	Master's Degree	Married	
6	26	Female	40000	Associate's Degree	Single	
7	31	Male	60000	Bachelor's Degree	Single	
8	36	Female	80000	Master's Degree	Married	
9	41	Male	105000	Doctorate	Single	
10	46	Female	90000	High School Diploma	Married	
11	51	Male	135000	Bachelor's Degree	Married	
12	27	Female	35000	High School Diploma	Single	
13	32	Male	55000	Associate's Degree	Single	
14	37	Female	70000	Bachelor's Degree	Married	

	Number of Children	Home Ownership	Credit Score
0	0	Rented	High
1	2	Owned	High
2	1	Owned	High
3	0	Owned	High
4	3	Owned	High
5	0	Owned	High
6	0	Rented	Average
7	0	Rented	Average
8	2	Owned	High
9	0	Owned	High
10	1	Owned	High
11	0	Owned	High
12	0	Rented	Low
13	0	Rented	Average
14	2	Owned	High

Total Records : 164

Target Column : Credit Score

Building Decision Tree...

MODEL TRAINING COMPLETED

Actual Values:

[0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 2, 1, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 2, 1, 0, 2, 0, 0, 0, 1, 1, 1, 0]

Predicted Values:

[np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(2), np.int64(1), np.

Accuracy:

94.0 %

Confusion Matrix:

```
[[ 9  2  0]
 [ 1 31  0]
 [ 0  0  7]]
```

Credit Score Classification Completed Successfully.